

Implementing Basic Linux Commands

Shell is a program that takes the commands you type and translates them into instructions to the operating system. A shell is a program that reads commands that are typed on a keyboard and then executes (i.e., runs) them. **Shells are the most basic method for a user to interact with the system.** There are different shells available; bash, zsh, fish, pwsh etc . In Ubuntu, the default shell is **bash**.

File: Under most operating systems (including Linux), there is the concept of a file, which is just a bundle of information given a name (called a **filename**). Examples of files might be your history term paper, an e-mail message, or an actual program that can be executed. Essentially, anything saved on disk is saved in an individual file.

Directory: With the concept of files comes the concept of directories. A **directory** is a collection of files. It can be thought of as a "folder" that contains many different files. Directories are given names, with which you can identify them. Furthermore, **directories are maintained in a tree-like structure; that is, directories may contain other directories.**

Root Directory: The directory system in linux starts from the root directory (/) which is represented by the sign of forward slash (/) and all the files/directories in your system reside either in your root or in the children-directories of root. Within root directory, is a directory named home (/home) , within that home directory is the directory of the User (/home/username/) which is also called as **User's Home directory**

Current working directory: At any moment, commands that you enter are assumed to be relative to your current working directory (until specified otherwise). You can think of your working directory as the directory in which you are currently "located". When you first log in and start your terminal, your working directory is set to your **User's home directory--/home/me.**

Basic syntax of Linux Commands

A command is an instruction given by a user telling a computer to do something, such as run a single program or a group of linked programs. Commands are generally issued by typing them in at the command line (i.e., the all-text display mode) and then pressing the ENTER key, which passes them to the shell.

Options(Flags/Switches) and Arguments: This brings us to a very important point about how most commands work. Commands are often followed by one or more options that modify their behavior, and further, by one or more arguments, the items upon which the command acts. So, most commands look kind of like this:

\$command -options arguments

Most commands use options/flags/switches consisting of a single character preceded by a dash/minus/heifen (-) , for example, “-r”.

You can also use multiple switches/options of a command together by simply writing one after the other.

\$command -option1 -option2 arguments

Basic Commands

In this section we will have an insight of some basic commands. Different commands take multiple arguments and options (where option starts with a dash - sign).

Command	Description	Options/Flags/Switches	Example
echo	The echo command prints its arguments to the standard output.		\$ echo Hello World Display “Hello World” on your terminal screen.
whoami	Displays the username who’s currently logged in		whoami
hostname	Displays the name of host machine		hostname
ls	List files and directories.	-R It is used to display contents recursively of all sub directories and their sub sub directories.	• ls Simply displays the contents of your directory. ls -R BNU It’ll display all contents of BNU and it’s sub directories till the end.
cd	Change directory.		cd /path/to/directory changes the current directory to the specified path.

pwd	Print current working directory.		pwd displays the current working directory.
mkdir	Create a new directory.	-p for creating hierarchy of nested directories	mkdir my_directory creates a new directory named “my_directory”. mkdir -p BNU/SCIT/SEM
rm	Remove files and directories.	-r: Remove directories recursively. * , wildcard { } , wildcard	rm file.txt deletes the file named “file.txt”. rm -r my_directory deletes the directory “my_directory” and its contents. rm -r * (removes everything in current working directory)
cp	Copy files and directories.	-r: Copy directories recursively.	cp -r directory destination copies the directory “directory” and its contents to the specified destination.

			cp file.txt destination copies the file “file.txt” to the specified destination.
mv	Move/rename files and directories. It is also responsible for hiding/unhiding files and directories	Simply rename a file/directory and start its new name with a . (dot) that file will be hidden. Similarly, rename that file/directory and remove the dot from starting to unhide that file/directory	mv file.txt new_name.txt renames the file “file.txt” to “new_name.txt”. mv file.txt directory moves the file “file.txt” to the specified directory. mv
touch	Create an empty file.	{01..10} Wild card to create multiple files	touch file.txt creates an empty file named “file.txt”. touch file{01..10}.txt Creates 10 empty .txt files.

tee	Read from standard input and write into files . Usually used by piping it with other commands	{01..10} Wild card	echo "hi" tee file{01..05}.txt
cat	View the contents of a file.		cat file.txt displays the contents of the file "file.txt".
head	Display the first few lines of a file.	-n: Specify the number of lines to display.	- head file.txt shows the first 10 lines of the file "file.txt". - head -n 5 file.txt displays the first 5 lines of the file "file.txt".
tail	Display the last few lines of a file.	-n: Specify the number of lines to display.	- tail file.txt shows the last 10 lines of the file "file.txt". - tail -n 5 file.txt displays the last 5 lines of the file "file.txt".
grep	Has a lot of uses but it mainly it is used to search for a pattern in texts. It returns the line where the pattern is found. Can be used to search for text in files, but can also be used to narrow down specific results from larger texts. Can be piped with other commands.	-i ignore case in pattern -v invert the results -n also display the line number where the pattern is found	grep "Ali" Names.txt grep -i "Ali" Names.txt (it'll ignore the case of Ali) grep -v "Ali" Names.txt (It'll return those lines which don't contain Ali) grep "Ali" Names.txt tee Ali.txt (it'll give the results of Ali to tee command which will write it in a file and also display it) grep "Ali" Names.txt wc -l (it'll count the no. of lines where our pattern is found)
date	Displays current time and date		date Shows date and time
script	Start recording your terminal session in a log file		script filename

			Start recording your session in that file
exit	Exit the script recording session. Also has a lot of other uses which we have not explored yet	Its behaviour changes depending on the context in which it is used	exit
clear	Clear the terminal screen	-x	clear -x clear the terminal screen but you can still scroll back to see your previous activity.
zip	for zipping a file/directory	-r for zipping directories -m for removing original copies of the file/directories which are being zipped.	zip path/zipfile-name.zip filepath/filename we can simply tell where to create the zip file and what should be the name of the zip file. We have to specify the name of the zip file. zip -m path/zipfile-name.zip filepath/filename It'll remove the original copies of the files being zipped zip -r path/zipfile-name.zip directory-path/directory-name
unzip	for extracting the zipped files.	-d we can specify where to extract the zipfile. By default, it extracts the contents of a zip file it in the current directory.	unzip zip-file-path/zip-name.zip It'll unzip the contents of this zip file in current directory unzip -d path/directory-name/ zip-file-path/zip-name.zip It'll unzip in our desired directory
nano	a simple text editor		nano filename.txt it'll simply open the file and we can edit that file and can simply save/unsaved changes and exit the text editor.

Output Redirection in Linux

1. > which we used with echo

2. >> we can also use this with echo or even other commands

To send the output of command to a file instead of the terminal, use the > redirection character:

```
$ command > file
```

```
echo "hi" > file.txt
```

The shell creates a file if it does not already exist. If a file exists, the shell erases the original file first.

You can append the output to the file instead of overwriting it with the » redirection syntax:

```
$ command » file
```

```
echo "hello" >> file.txt
```

Wild Cards

1 - {} can be used to perform bulk operations on files and directories.

examples :

```
touch filename-{01..10}.txt , mkdir folder{1,2,5,7,9,45,76}
```

2- * can be used to perform bulk operations on existing files/directories.

examples :

```
cp *.txt ./folder/ , rm -r name*.txt , rm name*rollnumber ,  
rm *Section-B*
```

Navigating Directories

Unix has a directory hierarchy that starts at /, usually called the root directory. The directory separator is also forward slash (/). Yes, it seems confusing at the start but with time you'll settle with it.

When you refer to a file or directory, you specify a path or pathname. There are two different ways of writing files path:

1. Absolute Path

When a path starts with the root directory, it is a full or absolute path.

/home/user/Desktop/Lab01

Similarly, we can also start a path with the User's home directory (denoted by ~)

~/Desktop/Lab01

2. Relative Path

A path starting at the current directory is called a relative path.

./Lab01 OR Lab01 (both are fine)

!!! /Lab01 is not fine in this case

/ refers to root directory

./ refers to the current directory

../ refers to the previous directory

~ refers to User's Home directory

